Práctica 03.
Aprendiendo Linux.
El Sistema de Ficheros



Autor: Francisco de Asís Conde Rodríguez / Lina García Cabrera
Copyright: Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional (CC BY-NC-ND 4.0)



nombres de ficheros, tipos de ficheros en linux, camino, ruta o trayectoria absoluta y relativa, diferencia entre directorio base y directorio de trabajo, listado largo, permisos en directorios y ficheros, cambiar permisos en formato simbólico o en formato octal con `chmod`, permisos preestablecidos `umask`, `setuid`, `setgid`, `Sticky bit`.

Contenidos

Contenidos **1**
1. Introducción **2**
1.1. Nombres de ficheros 3
1.2. Trayectorias de ficheros (paths) 4
1.3. Directorios base 7
2. Listado de ficheros con ls **8**
2.1. Opciones de la línea de órdenes 9
2.2. Interpretación del formato largo 10
2.3. Comodines en las trayectorias de ficheros 12
3. Cambio de directorio de trabajo **12**
4. Presentación del directorio de trabajo **13**
5. Jerarquía de directorios UNIX **13**
6. Permisos **17**
6.1. Permisos especiales de ficheros y directorios 18
6.2. Permisos de directorios 21
6.3. Establecimiento de permisos de acceso 22
6.4. Reducción de los permisos para ficheros de nueva creación (umask) 25
6.5. Ejemplos 26
7. Ejercicios propuestos **29**
Preguntas frecuentes **31**
Caracteres “comodín” ¿Qué son los caracteres "comodín"? 31
Octal ¿Qué es el sistema OCTAL? 33

1. Introducción

En este capítulo se utilizan algunos programas que puedes descargar desde aquí:

- cambia.cpp
- consultar.cpp

En Ubuntu 26.04 LTS. Muchas órdenes básicas (ls, cp, chmod, rm…) proceden de `uutils`, una reimplementación en Rust. Su comportamiento coincide con el descrito, pero algunos mensajes de error aparecen en inglés («Permission denied») y `ls -l` escribe el mes en inglés («Sep»). Los ejemplos de este guion se han obtenido en Ubuntu 24.04 LTS, la versión del aula.

El **sistema de ficheros de UNIX** consta de un conjunto de ficheros. Hay tres tipos de ficheros:

- **Ficheros ordinarios**, que contienen datos.
- **Directorios**, que contienen información sobre un conjunto de ficheros. Se emplean para localizar un fichero a partir de su trayectoria o ruta (path).
- **Ficheros especiales**, se utilizan para representar un dispositivo físico real como una impresora, una unidad de cinta o un terminal, utilizado para operaciones de entrada/salida (I/O). Los ficheros de dispositivos suelen estar en el directorio **`/dev`**. También tienen otros propósitos.

En los sistemas UNIX hay dos tipos de archivos especiales de dispositivo, **archivos especiales de caracteres** y **archivos especiales de bloque**:

- Cuando un archivo especial de caracteres se utiliza para la entrada/salida (I/O) del dispositivo, los datos se transfieren un carácter a la vez. Este tipo de acceso se llama acceso de dispositivo crudo (raw device access). Los terminales son dispositivos de carácter.
    - En la salida de formato largo de **`ls -l /dev`**, los archivos de caracteres especiales están marcados con el símbolo de la "**`c`**".

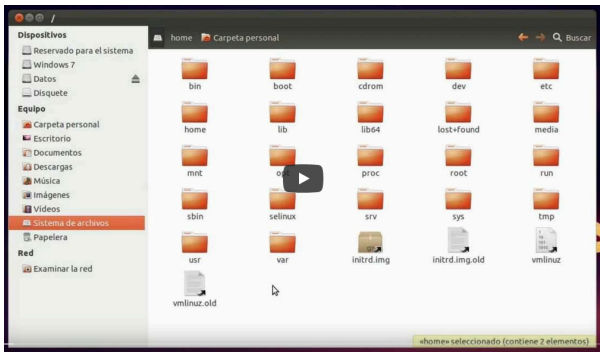
```
alumno@so-lab01:~$ ls -l /dev/tty /dev/tty0 /dev/tty1
crw-rw-rw- 1 root tty 5, 0 sep 28 16:37 /dev/tty
crw--w---- 1 root tty 4, 0 sep 28 16:37 /dev/tty0
crw--w---- 1 root tty 4, 1 sep 28 16:37 /dev/tty1
```

- Cuando se utiliza un archivo especial de bloque para la Entrada/Salida (I/O) del dispositivo, los datos se transfieren en grandes bloques de tamaño fijo. Este tipo de acceso se denomina acceso de dispositivo en bloque. Los discos son dispositivos de bloque.
    - En la salida de formato largo de **`ls -l /dev`**, los archivos especiales de bloque están marcados con el símbolo "**`b`**".

```
alumno@so-lab01:~$ ls -l /dev/sda*
brw-rw---- 1 root disk 8, 0 sep 28 16:37 /dev/sda
brw-rw---- 1 root disk 8, 1 sep 28 16:37 /dev/sda1
brw-rw---- 1 root disk 8, 2 sep 28 16:37 /dev/sda2
```

Al igual que la mayoría de los sistemas operativos modernos UNIX **organiza su sistema de ficheros como una jerarquía de directorios**, la cual muchas veces se denomina árbol, aunque casi siempre se dibuja como un árbol invertido. En la parte superior de la jerarquía está un **directorio especial: root (raíz)**.

Un directorio puede incluir ficheros normales y subdirectorios. UNIX considera a un subdirectorio como un fichero de tipo especial. Si se presenta el contenido de un directorio, se mostrarán los subdirectorios junto con los demás ficheros.



Vídeo 1. Estructura del sistema de archivos en Ubuntu (https://youtu.be/E_IHBKMrfOU)

1.1. Nombres de ficheros

Un **nombre de fichero** designa a un fichero dentro de un directorio, y consta de un máximo de 14 caracteres en la mayoría de los sistemas System V y un máximo de 255 caracteres en casi todos los sistemas basados en Berkeley. En Linux, con los sistemas de ficheros habituales (ext4), el límite es de 255 bytes (getconf NAME_MAX / devuelve 255); como en UTF-8 las vocales con tilde o la ñ ocupan 2 bytes, el número de caracteres puede ser menor.

El nombre de un fichero puede contener **cualquier carácter distinto de '/'**, aunque algunos caracteres como '-' y (espacio) causan problemas por su significado especial en la línea de órdenes. Casi siempre son caracteres seguros las letras (mayúsculas o minúsculas), los dígitos y los caracteres especiales (+=_:).

Por convención, un **punto al principio de un nombre de fichero** indica un **fichero de iniciación** (también se llama **fichero oculto**) u otro fichero de apoyo para un programa en particular. Los listados de directorios normalmente no incluyen los ficheros cuyos nombres comienzan por punto (se ocultan a menos que se acompañe la orden de algún parámetro, por ejemplo `ls -a`). Los archivos ocultos se utilizan para ejecutar algunos scripts o para almacenar la configuración de algunos servicios en su host. Además, los ficheros con punto reciben un trato especial durante la expansión con comodines.

Los nombres de ficheros **distinguen entre mayúsculas y minúsculas**, de forma que `datos`, `Datos` y `DATOS` nombran tres ficheros distintos.

Buenos consejos para nombrar archivos en Linux

Para evitar que se produzcan problemas con las rutas, caminos o trayectorias de los archivos, sigue estos buenos consejos:

- Nombrar todos sus archivos en minúsculas por defecto.
- En lugar de utilizar un espacio, utilice un (_).
- Usar tipos de archivo consistentes, siempre lo mismo. Por ejemplo, todos los archivos deben llevar la extensión del programa que lo genera.
- Sólo caracteres alfanuméricos, puntos, subrayados y guiones y no uses símbolos como "%", "$", etc.
- Mantén los nombres de los archivos cortos y descriptivos.

1.2. Trayectorias de ficheros (paths)

Una **trayectoria, camino o ruta** de fichero designa a un fichero dentro de la jerarquía de ficheros, y consta de una secuencia de nombres de fichero separados por diagonales (**/**). Hay dos tipos de trayectorias: las absolutas y las relativas.

Las trayectorias absolutas comienzan con una diagonal; no así las trayectorias relativas.

Trayectorias absolutas. Se puede hacer referencia a un fichero en cualquier parte del árbol especificando su trayectoria absoluta. Ésta contiene la secuencia de subdirectorios que se debe recorrer para llegar **de la raíz al fichero**. La diagonal que siempre inicia una trayectoria absoluta designa al directorio raíz.

Por ejemplo, UNIX interpreta la trayectoria **`/usr/humberto/ranas`** de la siguiente manera:

1. De la raíz, pasar al subdirectorio de nombre `usr`.
2. De este directorio, pasar al subdirectorio de nombre `humberto`.
3. En este subdirectorio escoger el fichero denominado `ranas` (el cual podría ser un subdirectorio).

Trayectorias relativas. Cada proceso tiene un directorio denominado **directorio actual** o **directorio de trabajo** (forma parte del BCP o descriptor de proceso, esa información se almacena en esa estructura de datos), el cual puede servir como punto de partida para las trayectorias de fichero. La orden **`pwd`** (***p**rint **w**orking **d**irectory)* presenta el directorio de trabajo del usuario o directorio actual, es decir, el directorio de trabajo de su proceso de shell. Una trayectoria que **no comienza con '/'** se denomina **trayectoria relativa**, y es relativa al **directorio de trabajo**.

Así pues, si el directorio de trabajo es **`/usr/humberto`**, la trayectoria relativa

```
viajes/ciudad.datos/jakarta
```

corresponde a la trayectoria absoluta

```
/usr/humberto/viajes/ciudad.datos/jakarta
```

El caso más común y sencillo de trayectoria relativa es un nombre de fichero. Este nombre de fichero se refiere a dicho fichero en el directorio actual.

Las notaciones '.' y '..'.

El **padre de un directorio** `d` (distinto del directorio raíz) es el directorio que está inmediatamente arriba de `d` en la jerarquía. El directorio actual se denomina '**`.`**' y su padre se llama '**`..`**', de manera que se pueden usar trayectorias para subir o bajar por la jerarquía.

Por ejemplo, si el directorio actual es **`/usr/humberto`**, como en el ejemplo anterior, '**`..`**' se refiere a **`/usr`**, '**`../..`**' se refiere a la raíz, y '**`../lara`**' se refiere a **`/usr/lara`**. El directorio raíz es un caso especial, ya que es su propio padre; así, '**`/..`**' se refiere a la raíz, lo mismo que '**`/`**'.

- **`.`** (punto simple) denota el directorio actual en la ruta.
- **`..`** (dos puntos) indica el directorio principal, es decir, un nivel superior.

¿Cuál debe utilizar? ¿Ruta relativa o ruta absoluta?

Para ser sinceros, no hay una respuesta directa a esta pregunta. Realmente depende de la situación.

Si estás muy abajo en la jerarquía de directorios y tienes que moverte un nivel hacia arriba o hacia abajo, el uso de la ruta relativa será más fácil.

Supongamos que estás en el directorio

```
/home/nombredeusuario/programación/proyecto/interfaz/src/header
```

y tienes que acceder a algo en el directorio

```
/home/nombredeusuario/programación/proyecto/interfaz/bin
```

El uso de la ruta relativa le ahorrará escribir todos esos largos nombres de directorios y puede usar simplemente

```
../../bin
```

Pero si tienes que acceder a algo en el directorio **`/usr/bin`** desde el directorio **`/home/nombredeusuario/programación/proyecto/interfaz/src/header`**,

usar algo como **`../../../../../../usr/bin`**

no será algo inteligente. Usar la ruta absoluta es lo más sensato en este caso.

Otro caso es el **uso de las rutas en scripts o programas**. Cuando estés seguro de la ubicación, **utiliza la ruta absoluta**.

¿Cuál sería la orden y la ruta absoluta o relativa para mostrar el contenido de **`mi_script.sh`**?

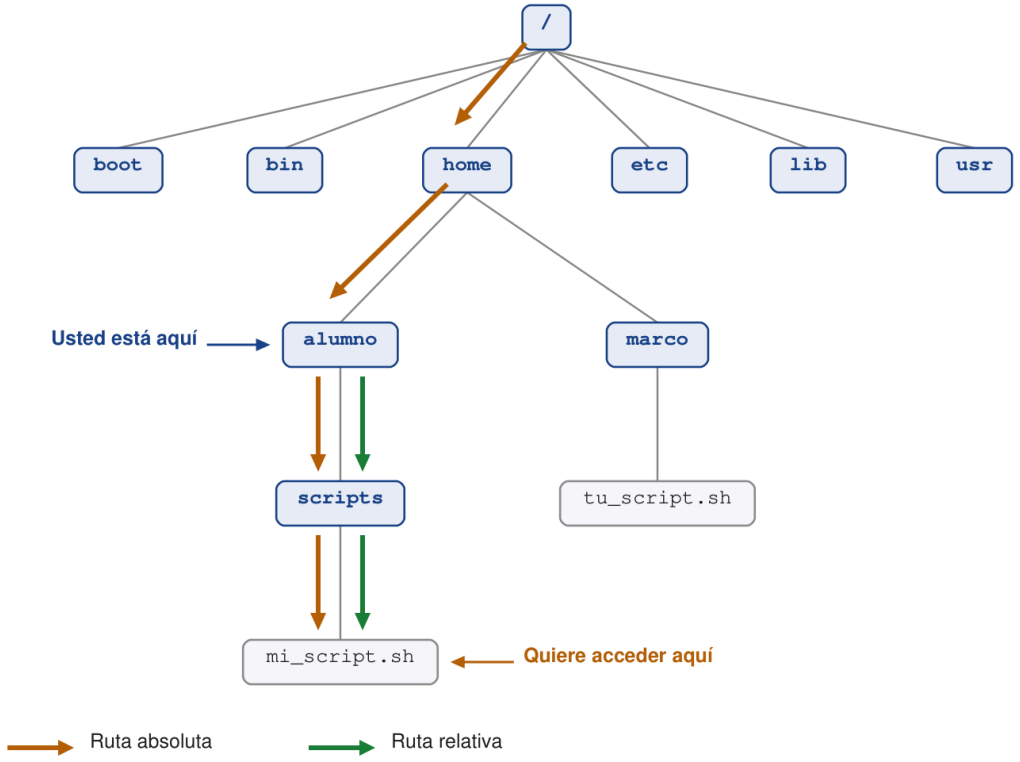


Ilustración 1. Ejemplo de ruta absoluta y relativa de un fichero

¿Cuál sería la orden y la ruta absoluta o relativa para cambiarte al directorio **`marco`**?

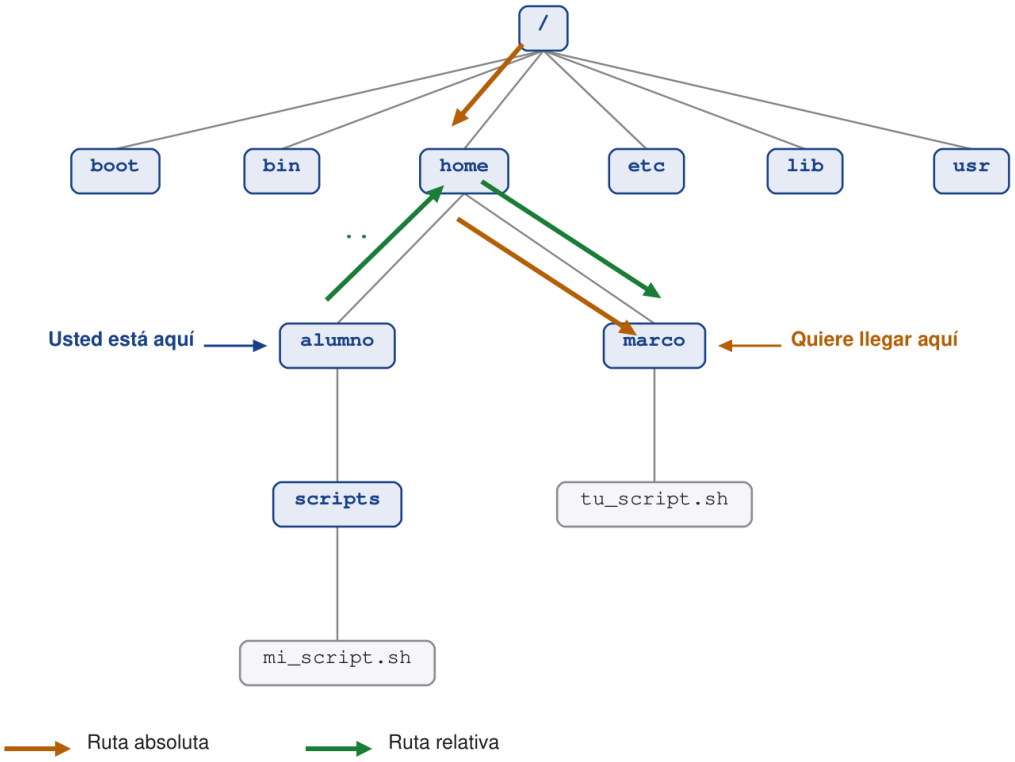


Ilustración 2. Ejemplo de ruta absoluta y relativa de un directorio

1.3. Directorios base

Todo usuario tiene un **directorio base** (***home directory***). La trayectoria absoluta de ese directorio varía de acuerdo con el sistema, pero su último componente generalmente es el nombre del usuario. En System V, el directorio base del usuario `usuario` casi siempre es el directorio **`/usr/usuario`**, y en BSD UNIX normalmente es `/u/usuario`. Otros sistemas usan **`/home/usuario`** o **`/u/instalación/usuario`**, donde `instalación` es el nombre del ordenador anfitrión. El directorio base de un usuario se guarda en la **variable de entorno `HOME`**. Puede mostrar el valor de la variable `HOME`, ejecutando **`echo $HOME`**. En Ubuntu los directorios base son subdirectorios de /home (por ejemplo, /home/alumno); los ejemplos siguientes conservan el convenio de System V, en el que lo eran de /usr.

Nota: las variables de entorno son pares clave-valor que contienen información sobre el sistema y el entorno del usuario, influyendo en el comportamiento de los procesos y aplicaciones. Para verlas, se utilizan los comandos **`printenv`** y **`env`**, que muestran la lista completa de variables, o **`echo $NOMBRE_VARIABLE`** para ver el valor de una variable específica, como **`echo $HOME`** o **`echo $PATH`**.

Los directorios base son muy importantes, por lo que hay una notación especial que reconocen todos los shells de UNIX, con la excepción del shell Bourne original (dash, el intérprete sh de Ubuntu, sí la reconoce): el **metacarácter '~' al inicio de una trayectoria se**

refiere al directorio base del usuario actual, mientras que '~' antes de nombre de usuario se refiere al directorio base de ese usuario.

Si el directorio base en un sistema Unix es **`/usr/nombredeusuario`**. Entonces, para el usuario `humberto` '**`~/helado`**' se refiere a **`/usr/humberto/helado`** y para el usuario `zelda` '**`~/galletas`**' se refiere a **`/usr/zelda/galletas`**.

por ejemplo

```
alumno@so-lab01:~$ echo ~
/home/alumno
alumno@so-lab01:~$ echo ~root  # Directorio base de root
/root
```

2. Listado de ficheros con ls

A lo largo de las prácticas hemos utilizado muchas veces esta orden. En esta sección la estudiaremos con algo más de detalle.

La orden **`ls`** lista:

- un conjunto de ficheros,
- el contenido de un directorio,
- el contenido de un árbol de directorios
- o cualquier combinación de los anteriores.

Puede emplearse para ver si existen ficheros o para examinar sus características.

Su sintaxis es (recuerda que puedes ejecutar **`man ls`** para conocer todas la opciones):

```
$ ls [opciones] nombre ...
```

Esta orden lista los directorios y ficheros especificados en `nombre ...`, y el estilo del listado lo determinan las opciones o el valor por omisión. Dos conjuntos de opciones especialmente útiles son **`-CF`** y **`-l`**. La opción **`-l`** ya la conocemos y la orden

```
$ ls -CF nombre ...
```

lista los ficheros en formato de columna e indica cuáles son directorios o ficheros ejecutables.

Cada nombre en `nombre ...` es un directorio o un fichero; los nombres sucesivos se separan con espacios. Si se omite `nombre ...`, se considera como '.' (el directorio actual). Se pueden usar metacaracteres mágicos (comodines) para especificar varios ficheros (ver la sección 2.3). Los elementos que se listarán se determinan de la siguiente manera:

- Si un nombre designa un fichero que no es un directorio, se presenta ese fichero.
- Si el nombre designa un directorio, se listan los ficheros del directorio.

- Si no hay ningún fichero con ese nombre, no se lista nada.

Se excluyen los ficheros y directorios cuyos nombres comienzan con punto, a menos que se especifique la opción **`-a`**.

Un ejemplo de utilización de **`ls`** es:

```
alumno@so-lab01:~$ ls -Fl -a .. harpo
```

Esta orden lista dos conjuntos de ficheros: aquellos en el directorio padre del directorio actual y aquellos en harpo. Si harpo es un subdirectorio del directorio actual, se listan los ficheros en dicho subdirectorio; si harpo es un fichero del directorio actual, sólo se lista el fichero; y si harpo no está en el directorio actual, no se lista nada por harpo. Las opciones de listado que se aplican son **`-F`**, **`-l`** y **`-a`**.

2.1. Opciones de la línea de órdenes

La orden **`ls`** puede producir dos tipos de listados: corto y largo.

- El **listado corto,** que se presenta por omisión, incluye el nombre de cada fichero y la demás información que se solicite de forma explícita;
- el **listado largo** incluye los permisos de fichero e información adicional. La opción **`-l`** genera un listado largo, si no está presente se obtiene un listado corto. A continuación comentamos las opciones más útiles.

Las opciones que siguen afectan al conjunto de ficheros que se incluyen en el listado:

- **`-a`** Lista todos los ficheros o directorios, incluyendo aquellos cuyo nombre comienza con punto, incluidas las entradas «.» y «..».
- **`-d`** Si un nombre de la línea de órdenes designa un directorio, **sólo se lista el nombre del directorio** y no su contenido. Se puede usar esta opción junto con -l para ver el estado de un directorio.
- **`-R`** Lista los subdirectorios de forma recursiva. En otras palabras, para cada subdirectorio que **`ls`** encuentra lista los ficheros de ese subdirectorio y de cada uno de los subdirectorios que contiene directa o indirectamente. El listado muestra la organización de los subdirectorios.
- **`-f`** Esta opción sólo se aplica a directorios y hace que `ls` liste todos los ficheros que contiene el directorio en el orden en que se almacenaron. Esta opción anula las opciones **`-l, -t, -s y -r`** (si se especifican) e implica la opción **`-a`** para los ficheros que contiene el directorio.

Las opciones que siguen afectan a **la información relacionada con los ficheros** que se presentan en un listado, sea largo o corto:

- **`-F`** Coloca una **`'/'`** después de cada fichero que es un directorio y un ‘*’ después de cada fichero ejecutable.

- **`-i`** Muestra el número de nodo i de cada fichero (veremos los nodos i en teoría, es un estructura de datos con las propiedades y forma de localizar el contenido del fichero).
- **`-s`** Muestra el tamaño en bloques de cada fichero, incluyendo los bloques indirectos (bloques que contienen punteros a otros bloques).

Las opciones siguientes **controlan la distribución de un listado corto**:

- **`-C`** Produce una salida de varias columnas, con las entradas clasificadas de forma vertical.
- **`-x`** Produce una salida de varias columnas, con las entradas clasificadas de forma horizontal.
- **`-m`** Lista los ficheros a lo ancho de la página, separando los ficheros sucesivos con comas. Esta opción no está disponible en algunos sistemas.

Las opciones que siguen afectan al **orden de presentación de los ficheros**:

- **`-t`** Lista los nombres en orden cronológico, comenzando por los más recientes (a menos que se invierta el orden con **`-r`**). Para la clasificación se utiliza la hora de la última modificación, a menos que se especifique la opción **`-u`**.
- **`-r`** Lista los ficheros en orden alfabético inverso si no existe la opción **`-t,`** y en orden cronológico de más antiguo a más nuevo si está presente la opción **`-t`**.

Las opciones que se presentan a continuación **afectan a la hora asociada a cada fichero**, y la hora afecta al orden de los ficheros si se ha especificado la opción **`-t`**; también aparece en los listados largos incluso si no se ha especificado **`-t`**. Por omisión, la hora es la de la última modificación.

- **`-u`** Usa la hora del último acceso al fichero.
- **`-c`** Usa la hora de la última modificación del nodo i del fichero, es decir, la última vez que cambió la información de estado del fichero (permisos, propietario, número de enlaces) o su contenido; no es la fecha de creación. Si están presentes tanto **`-u`** como **`-c`**, **`-u`** controla la hora.

2.2. Interpretación del formato largo

La orden **`ls`** produce un listado largo si se especifica la opción **`-l`**. A continuación se presenta un ejemplo del contenido de un directorio listado con las opciones **`-laF`**:

```
alumno@so-lab01:~/ejemplo$ ls -laF
total 20
drwxrwxr-x 3 alumno alumno 4096 sep 28 16:37 ./
drwxr-x--- 9 alumno alumno 4096 sep 28 16:37 ../
drwxrwxr-x 2 alumno alumno 4096 sep 28 16:37 dl/
-rwxr-xr-x 1 root   root     10 sep 28 16:37 loockup*
lrwxrwxrwx 1 alumno alumno   10 sep 28 16:37 mark -> tmp/marvin
-rw-rw-r-- 3 alumno alumno    5 sep 28 16:37 news
prw-rw-r-- 1 alumno alumno    0 sep 28 16:37 tubo|
```

El listado se interpreta de la siguiente manera:

- El primer carácter de la línea indica el tipo de fichero:
  - **- Fichero ordinario**
  - **d Directorio**
  - **l Enlace simbólico** (lo veremos en el siguiente tema). La entrada de un enlace simbólico muestra su nombre en el directorio actual y el nombre del fichero con el que está enlazado.
  - **b Dispositivo de bloques**, se accede a ellos por bloques de tamaño fijo, en cualquier orden, a través de la caché de bloques (buffer) del sistema operativo; por ejemplo, los discos.
  - **c Dispositivo de caracteres**, se accede a ellos como a una secuencia de bytes, sin pasar por la caché de bloques; por ejemplo, los terminales o /dev/null.
  - **p Interconexión** (tuberías, cauce en inglés **pipe**) con nombre. Las **tuberías sin nombre** tienen asociado un fichero en memoria principal, por lo tanto, son temporales y se eliminan cuando no están siendo usadas ni por productores ni por consumidores. Permiten la comunicación entre el proceso que crea un cauce y procesos hijos tras la creación de la tubería. Las **tuberías con nombre** el cauce se crea en el sistema de archivos, y por lo tanto no tienen carácter temporal. Se manejan mediante llamadas al sistema (open, close, read y write) como el resto de ficheros del sistema. Permiten la comunicación entre los procesos que usen dicha tubería, aunque no exista una conexión jerárquica entre ellos.
- Las nueve letras siguientes indican los **permisos del fichero** como veremos en este mismo tema.
- El elemento que sigue indica cuántos **enlaces duros** tiene ese fichero (lo veremos también en el siguiente tema). **Un directorio dir generalmente tiene varios enlaces, ya que cada uno de sus subdirectorios incluye una entrada para dir como directorio padre y esta entrada cuenta como enlace a dir**.
- Los dos elementos que siguen indican el **usuario y el grupo propietarios del fichero**. El grupo usualmente es el del usuario, pero no es obligatorio que así sea.
- El elemento que sigue muestra el tamaño del fichero en bytes**.** En la parte superior se presenta el número total de bloques de disco ocupados por los ficheros del listado; en el ejemplo se ha ejecutado con la opción -a y se incluyen también los bloques que ocupan . y .., en este caso las 7 entradas del listado ocupan 20 bloques (en Linux, `ls` expresa el total en unidades de 1024 bytes).
- Los dos elementos que siguen indican **la fecha y la hora de la última modificación del fichero**. Si se usó la opción **`-u`**, se presenta la hora del último acceso; si se empleó la opción **`-c`**, aparece la hora de la última modificación del nodo i (ver las descripciones de estas opciones en la sección anterior).
- El último elemento **muestra el nombre**. Si hay una '**/**' después del nombre del fichero, indica que se trata de un directorio; un '*****' indica un fichero ejecutable (basta con que tenga una x en cualquiera de los tres grupos de permisos).

2.3. Comodines en las trayectorias de ficheros

Como vimos en el primer tema los shells permiten hacer referencia a un conjunto de ficheros usando una trayectoria de fichero que contenga uno o más comodines. En esta sección estudiaremos rigurosamente los tipos de comodines y su interpretación.

Hay tres notaciones para los comodines:

- El carácter **`*`** representa **cualquier secuencia de cero o más caracteres.**
- El carácter **`?`** **representa un sólo carácter**.

La construcción **`[conjuntoc]`** representa cualquier carácter del conjunto **`conjuntoc`**. El conjunto `conjuntoc` se escribe como una secuencia de caracteres y pares de caracteres. Un par de caracteres tiene la forma '**c1-c2**' y representa **los caracteres entre c1 y c2** en el conjunto de caracteres ASCII. Tres pares de caracteres de utilidad especial son '**`a-z`**' (las letras minúsculas), '**`A-Z`**' (las letras mayúsculas) y '**`0-9`**' (los dígitos decimales). Si se coloca el símbolo **`!`** antes de la secuencia, indica todos los caracteres que no están en la secuencia (algunos shells no permiten la utilización de !). A una '**`/`**' en la trayectoria real del fichero debe corresponder una '/' explícita en el patrón. Además, un '**`.`**' en la trayectoria real de un fichero que esté al principio de la trayectoria o inmediatamente después de una '/' debe tener un '**`.`**' correspondiente en el patrón.

A continuación se presentan algunos ejemplos de los comodines y los ficheros a los que pueden referirse:

Nombre	Ficheros
`gn*.l`	*gnu.l*, *gneiss.l*, *gn.l*, pero no *gn/x.l*
`~/.[a-zA-Z]*`	*~/.login*, *~/.mailrc*, pero no *~/login*
`*/hacer*`	*uno/hacer*, *dos/hacer.c*, pero no *hacer*
`zz?`	*zz1*, *zza*, pero no *zz12*
`[A-Z]*[!0-9]`	*Ala*, *Mario*, pero no *rosca* o *Rosca0*
`*.[acAC]`	*archivo.a*, *archivo.C*, pero no *.a*

Puedes encontrar más información sobre los Caracteres Comodín en Preguntas Frecuentes (Prácticas Sistemas Operativos).Caracteres “comodín” ¿Qué son los caracteres "comodín"?

3. Cambio de directorio de trabajo

La orden **`cd`** sirve para cambiar el directorio de trabajo. Su sintaxis es:

```
$ cd [dir]
```

donde **dir** es la trayectoria (absoluta o relativa) del nuevo directorio actual. **Si se omite dir se cambia el directorio actual al directorio base** (también equivale a *cd ~*).

cd es una orden que implementa el shell. Esto es así porque el directorio actual es una propiedad de los procesos. Si el shell creara un hijo para que cambiara de directorio actual sólo se cambiaría en el hijo.

4. Presentación del directorio de trabajo

La orden **pwd** (***P**rint **W**orking **D**irectory*) envía (muestra) la trayectoria del directorio de trabajo a la salida estándar (normalmente la pantalla). Atención: la orden interna de bash usa por omisión -L, mientras que el programa /usr/bin/pwd usa -P si no se indica ninguna opción.

Opción	Función
-L **--logical**	Obtiene la ruta del directorio actual mostrando el enlace simbólico (opción por defecto)
-P **--physical**	Obtiene la ruta física o real del directorio resolviendo el enlace simbólico

5. Jerarquía de directorios UNIX

En esta sección vamos a explicar algunos directorios de UNIX que guardan ficheros que forman parte importante del entorno proporcionado por UNIX. Sería útil que el lector explorara dichos directorios con órdenes como:

```
alumno@so-lab01:~$ ls -l /usr/bin | more
```

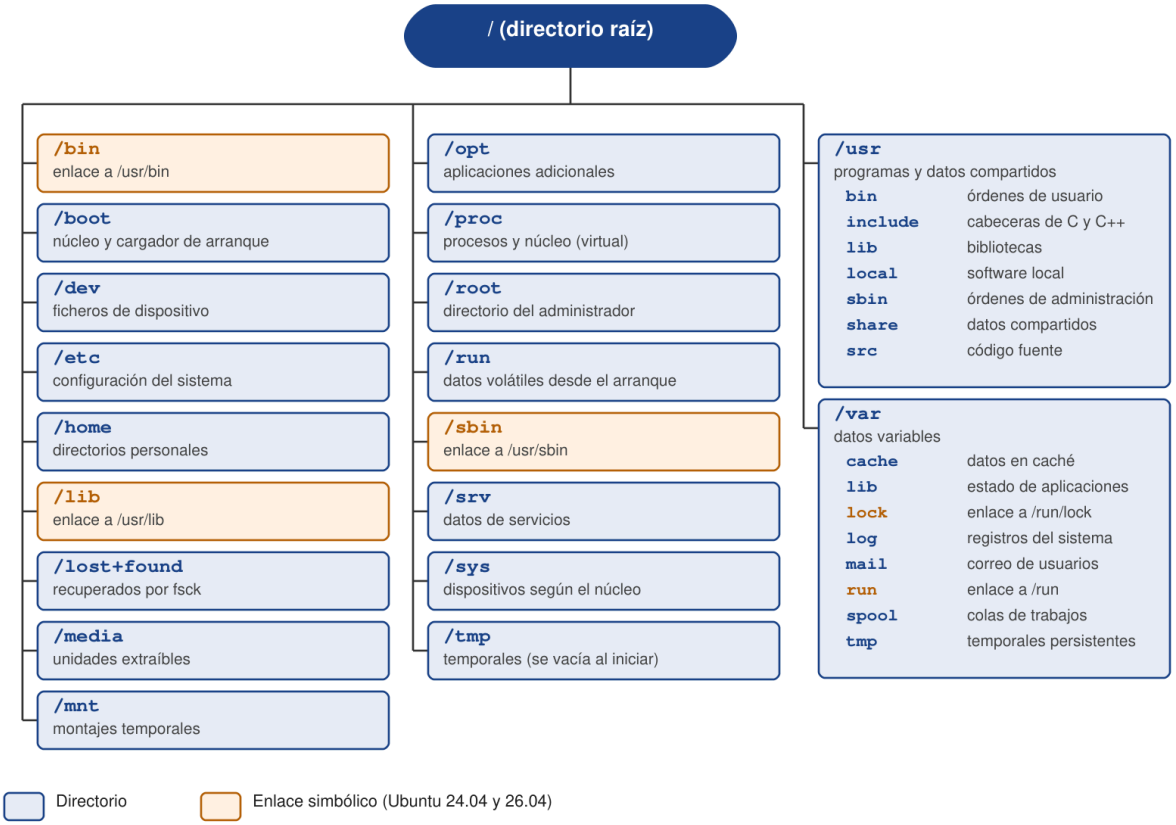


Ilustración 3. Ejemplo de estructura de directorios en un sistema Ubuntu

En teoría los directorios de UNIX pueden tener cualquier nombre. En la práctica los nombres y la utilización de varios directorios de primer y segundo nivel de los sistemas UNIX están firmemente establecidos por la tradición.

- **`/bin`**: Contiene ciertos programas, disponibles para todos los usuarios, que forman parte de la distribución normal de UNIX. Generalmente el directorio **`/bin`** incluye programas como **`ls`**, esenciales para iniciar un sistema y repararlo cuando hay problemas. Los ficheros ejecutables especializados pueden estar en otros directorios, la mayoría de los cuales tienen **`'bin'`** en sus trayectorias (**`'bin'`** viene de *binary*, binario). En Ubuntu, /bin es un enlace simbólico a /usr/bin.
- **`/boot`**: Contiene todos los ficheros necesarios para que arranque el sistema Linux, incluyendo la imagen binaria del kernel de LINUX. El nombre del fichero del kernel es vmlinuz, seguido por la información de versión y de edición. Por ejemplo, en Ubuntu 24.04, el kernel se encuentra en el fichero `/boot/vmlinuz-6.14.0-32-generic`.
- **`/dev`**: Contiene ficheros correspondientes a dispositivos externos, como impresoras (**`'dev'`** viene de *device*, dispositivo). Están divididos en dos grupos: ficheros especiales de caracteres y de bloques. Los primeros corresponden a dispositivos cuya E/S se hace en caracteres como el teclado. Los de bloques llevan a cabo la E/S en términos de bloques de bytes, como el disco duro.
- **`/etc`**: Contiene la mayoría de los archivos de configuración del sistema y las aplicaciones. Por ejemplo, ficheros como el de cuentas de usuario (**`/etc/passwd`**)

que se requieren para la administración del sistema. Según el estándar de jerarquía de ficheros (FHS), /etc no debe contener programas ejecutables. También se encuentra aquí **`/etc/group`**, que almacena los miembros de cada grupo.

- **`/home`**: Contiene los directorios iniciales de los usuarios. En ubuntu es el directorio que contiene los **directorios base de los usuarios**.
- **`/lib`**: Contiene bibliotecas (*libraries*) de subprogramas C compilados, que los programas C pueden llamar y enlazar.
- **`/lost+found`**: Contiene todos ficheros del sistema que no están asociados a ningún directorio. Sirve como un "cajón de sastre" para almacenar archivos y directorios que el sistema recupera tras un fallo o una corrupción del sistema de archivos. Estos ficheros se encuentran ejecutando la orden fsck (comprobación del sistema de ficheros).
- **`/mnt`**: Sirve como un punto de montaje temporal para sistemas de archivos. Lo utiliza el administrador del sistema para montar temporalmente sistemas de ficheros con la orden **`mount`**. Cuando se montan particiones de disco, unidades de red, dispositivos USB y otros medios. Aunque los medios extraíbles, como las unidades USB, suelen montarse automáticamente en directorios bajo **`/media`** en sistemas modernos.
- **`/opt`**: Sirve para instalar software adicional o de terceros que no siguen la estructura estándar del sistema de archivos. Los programas deben ubicarse en `/opt/nombre_paquete/bin`, en donde `nombre_paquete` es el nombre del paquete instalado.
- **`/proc`**: contiene un sistema de archivos virtual que no ocupa espacio en disco (el núcleo genera su contenido en el momento de leerlo) y que proporciona información sobre el kernel, los procesos y el hardware de un sistema Linux. En su nivel superior, contiene directorios numéricos que corresponden a los ID de procesos (PIDs) y ficheros con información del sistema, como el estado de la red (/proc/net) o la CPU (/proc/cpuinfo).
- **`/root`**: Se utiliza en muchos sistemas LINUX como directorio inicial de la cuenta root, está protegido.
- **`/usr`**: El nombre procede de user (usuario), porque en los primeros UNIX contenía los directorios de los usuarios; la expansión "User System Resources" es una interpretación posterior. Contiene la mayoría de los programas, bibliotecas, archivos de documentación (manuales) y otros datos de solo lectura que no son esenciales para el sistema en sí, pero que los usuarios utilizan para sus actividades diarias. También contiene subdirectorios importantes que se presentan a continuación. La estructura de subdirectorios de **`/usr`** imita parcialmente la estructura de root.
    - **`/usr/bin`**: Por lo general contiene ficheros ejecutables de utilidad, como el editor vi, órdenes como ls, cat, mkdir, que no son indispensables para la operación del sistema. Los ficheros ejecutables locales de su instalación podrían estar en `/usr/bin`, pero es más probable que estén en otro directorio como `/usr/local/bin`.
    - **`/usr/lib`**: Contiene diversas bibliotecas y subdirectorios de bibliotecas especializadas para recursos como el sistema X Windows.

  - **`/usr/src`**: Contiene código fuente, por ejemplo las cabeceras del núcleo que necesitan algunos controladores. En Ubuntu suele estar vacío: el código fuente de los paquetes se obtiene con apt source.
  - **`/usr/include`**: Contiene ficheros de cabecera para programas en el lenguaje de programación C y C++.
  - **`/usr/adm`**: Contiene información contable, ficheros de diagnóstico generados en el momento de una avería del sistema e información similar requerida por los administradores del sistema. Es un directorio histórico que no existe en Ubuntu; su función la cumple **`/var/log`**.
  - **`/usr/spool`**: Contiene los ficheros de colas de impresión, es decir, ficheros temporales en espera de procesamiento. Incluye los ficheros por imprimirse, bitácoras administrativas, ficheros que se enviarán a otros ordenadores y ficheros recibidos de otros ordenadores que deben ser recogidos por los usuarios. Los ficheros en `/usr/spool`, a diferencia de los de `/tmp` (descrito más adelante) siempre se conservan, incluso si el sistema se apaga. El directorio **`/usr/spool` es un directorio obsoleto** y no se usa comúnmente en sistemas Linux modernos. En su lugar, el directorio **`/var/spool`** ahora contiene los datos en cola de diversas aplicaciones, como correos electrónicos del sistema y colas de impresión. El nombre "spool" se refiere a "*simultaneous peripheral operations online*", indicando la gestión de tareas como la impresión
  - **`/usr/tmp`**: Una alternativa de /tmp, descrito más adelante. No existe en Ubuntu; su función la cumple **`/var/tmp`**.
  - **`/usr/ucb`**: Contiene programas específicos de Berkeley, como vi y mail (incluso en algunos sistemas que no son de Berkeley). En la mayoría de los sistemas Linux modernos, la mayoría de estos componentes se encuentran en `/bin`, `/usr/bin` o `/usr/local`. No existe en Ubuntu.
- **`/tmp`**: Contiene archivos y directorios temporales creados por el sistema operativo y las aplicaciones para almacenar datos que se necesitan solo por un corto período de tiempo. Estos archivos se suelen eliminar automáticamente al reiniciar el equipo o cuando ya no son necesarios, liberando así espacio en el sistema

En Ubuntu 26.04 LTS. `/tmp` es un sistema de ficheros en memoria (tmpfs): su contenido se pierde siempre al apagar el equipo. Para datos temporales que deban sobrevivir a un reinicio se usa `/var/tmp`.

- **`/var`**: Contiene archivos y datos del sistema operativo y aplicaciones que cambian durante su funcionamiento normal, como registros del sistema (`/var/log`), archivos de la cola de impresión (`/var/spool`), caches y bases de datos temporales, y otros archivos de datos que se modifican, añaden o eliminan con el tiempo. En los sistemas actuales, `/var/log` y `/var/spool` sustituyen a los antiguos /usr/adm y /usr/spool.

Esta estructura de directorios generalmente se duplica en otros directorios. Por ejemplo, el usuario `chico` podría mantener sus programas personales en `/usr/chico/bin` y los ficheros fuente de estos programas en `/usr/chico/src`.

6. Permisos

Cuando un usuario entra en una sesión, teclea un nombre de usuario, y después confirma que efectivamente es esa persona tecleando una clave. El nombre es la identificación para iniciar la sesión del usuario, o *login-id*. Pero el sistema en realidad reconoce al usuario por medio de un número, llamado su **identificador de usuario** o **uid** (*User IDentifier*). Además, al usuario se le asigna un **identificador de grupo** o **gid** (*Group Identifier*), que lo sitúa en una clase de usuarios. En Ubuntu, cada usuario tiene su propio grupo principal, con su mismo nombre: el usuario alumno pertenece al grupo alumno (grupos privados de usuario). El sistema de ficheros y, por tanto, el sistema UNIX, determinan, en general, lo que el usuario puede hacer con los ficheros mediante los permisos otorgados a su **`uid`** y a su **`gid`**.

El fichero `/etc/passwd` es el "fichero de claves o contraseñas". Contiene toda la información de inicio de sesión de cada usuario. El lector puede descubrir su `uid` y su `gid` como lo hace el sistema, buscando su nombre en **`/etc/passwd`**:

```
alumno@so-lab01:~$ grep alumno /etc/passwd
alumno:x:1000:1000:Alumno,,,:/home/alumno:/bin/bash
```

La orden **`grep`** será estudiada más adelante, su función es seleccionar las líneas de un fichero de texto que contienen un patrón de búsqueda (en este caso alumno). Los campos del fichero de claves están separados por "**`:`**", y están ordenados como sigue:

```
nombre de usuario:x (clave):uid:gid:varios:directorio base:shell de inicio
```

El fichero `/etc/passwd` es un fichero de texto normal, pero el contenido, el orden de aparición de los campos y los separadores son una convención reconocida por los programas que usan la información del fichero.

El campo de shell de inicio está a menudo vacío, lo que quiere decir que se usa el shell por defecto. El campo varios puede contener cualquier información, incluye con frecuencia el nombre completo del usuario.

El sistema operativo decidió permitir al usuario que leyera `/etc/passwd`, consultando los permisos asociados con el fichero. Existen tres tipos de permisos para cada fichero:

- **`r`** (read) leer (es decir, examinar su contenido),
- **`w`** (write) escribir (cambiar su contenido, distinto a borrar el fichero) y
- **`x`** (execute) ejecutar (ejecutarlo como programa).

Además, se puede aplicar un permiso diferente a cada persona. **Como propietario de un fichero**, el lector posee un conjunto de permisos de lectura, escritura y ejecución. Su "**grupo**" tiene otro conjunto. Los demás (a los que llamaremos "**otros**") tienen un tercer conjunto.

La opción **`-l`** de la orden ls despliega información, entre otras cosas, sobre los permisos:

```
alumno@so-lab01:~$ ls -l /etc/passwd
-rw-r--r-- 1 root root 940 sep 28 16:37 /etc/passwd
```

Esta línea puede interpretarse como sigue: `/etc/passwd` pertenece al usuario `root`, del grupo root, su tamaño es de `940` bytes, fue modificado por última vez el 28 de septiembre a las 16:37 y posee un enlace (hablaremos sobre enlaces más adelante).

La cadena **`-rw-r--r--`** es la manera en que **`ls`** representa los permisos del fichero.

- El primer **-** indica que se trata de un fichero ordinario. Si se tratara de un directorio habría una **`d`** en su lugar.
- Los tres caracteres siguientes codifican los **permisos de lectura, escritura y ejecución del dueño o propietario del fichero** (basados en el `uid`). **`rw-`** significa que `root` (el propietario) puede leer o escribir sobre el fichero, pero no ejecutarlo. Un fichero ejecutable tendría una **`x`** en vez de un guion.
- Los tres caracteres siguientes (**`r--`**) codifican los **permisos del grupo**; en este caso significa que las personas que pertenecen al grupo `root`, pueden leer el fichero, pero no escribir en él, ni tampoco ejecutarlo.
- Los tres siguientes (también **`r--`**) definen los permisos para cualquier otra persona (el resto de los usuarios del sistema que no son el propietario ni los que pertenecen al grupo).

En esta máquina, por lo tanto, sólo `root` puede cambiar la información de inicio de sesión para un usuario, pero cualquiera puede leer el fichero para descubrirla.

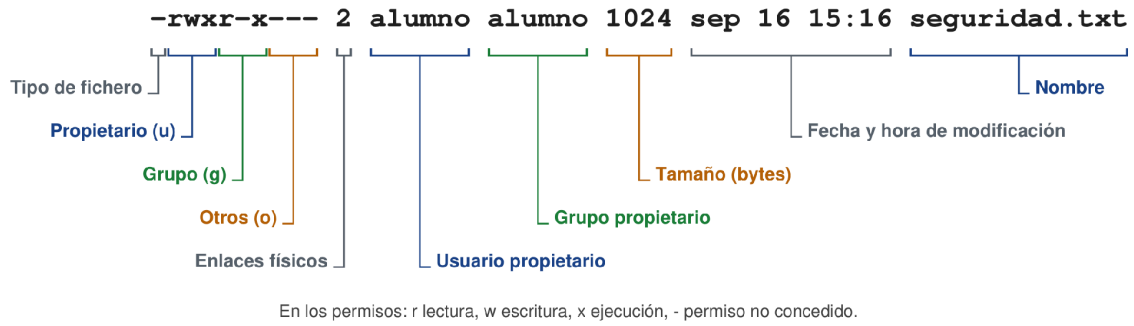


Ilustración 4. Significado de los campos de una línea de ls -l

6.1. Permisos especiales de ficheros y directorios

Además de los bits de permisos **`rwx`**, UNIX tiene otros tipos de permisos de fichero que usan principalmente (pero no exclusivamente) los programadores de sistemas: **Sticky Bit**, **set-uid bit** y **set-gid bit**. Estos 3 permisos especiales pueden modificar la forma en que funciona un directorio o cómo se ejecuta un programa. Se pueden especificar en modo simbólico o numérico.

Estos agregan un **carácter más al código**, a la nomenclatura octal con cuatro números (`xxxx`), pero no a la simbólica (–). Por ejemplo, podrías encontrarte con códigos tipo `4777`,

`rwS`, etc. El primer número en octal se utiliza para activar estos permisos, valor octal **1 para sticky bit**, valor octal **2 para set-gid bit** y valor octal **4 para set-uid bit**. Los valores siempre pueden ser `-, S, s, t, T, x`, pero no varios a la vez.

A continuación describimos algunos de estos permisos.

El bit set-uid

El bit set-uid otorga a un programa los permisos del propietario, y no los del usuario que lo llama. En un listado este bit aparece como una **`s`** en lugar de la `x` en los permisos del propietario, o el valor octal 4. Se especifica con una **`s`** al usar la orden **`chmod`**.

Normalmente, cuando se llama a un programa, y éste tiene acceso a un fichero, los privilegios de acceso del programa al fichero son aquellos relacionados con el identificador del usuario que llamó al programa.

Como ejemplo, suponga lo siguiente:

- El usuario `groucho` es dueño del fichero ejecutable `juego`.
- El programa `juego` utiliza el fichero de datos `records`, que pertenece a `groucho`.
- El usuario `homer` ejecuta el programa `juego`.

Los permisos que se aplicarán a `records` durante esta ejecución son los de "`otros`" (ya que `homer` no es dueño de `records`). A menos que `groucho` otorgue a "otros" el permiso de escritura de `records`, el programa `juego` no podrá modificarlo. Sin embargo, si confiere dicho permiso de escritura, `homer` podrá modificar directamente el fichero `records` y hacer lo que quiera con la información que contenga, utilizándola de manera no contemplada por `groucho`.

El bit *set-uid* permite evitar esta situación. Al activarlo en un fichero ejecutable se especifica que durante la ejecución del programa correspondiente los permisos aplicables son los del propietario del programa, no los del usuario que lo llama. Es decir, el bit *set-uid* confiere permiso al usuario para tener acceso indirecto a un fichero como `records`, a través de `juego`, pero no de forma directa.

Por ejemplo, suponga que se añade lo siguiente al ejemplo anterior:

- Los permisos de `records` para el dueño son rw.
- Los permisos de `records` para "`otros`" están desactivados.
- El bit set-uid del fichero `juego` está activo.

Ahora, cuando `homer` ejecuta `juego`, el proceso se comporta como si el usuario fuera `groucho`, no `homer`. Por consiguiente, el proceso tiene el permiso de escritura necesario para `records`, pero `homer` no puede tener acceso de lectura o escritura a `records` de forma directa.

El ejemplo clásico de utilización del bit **set-uid** es la orden **`passwd`**. El fichero de claves `/etc/passwd` pertenece a `root` (el superusuario, hacer `ls -l /etc/passwd`), y la orden **`passwd`** puede modificar el fichero `/etc/passwd`. Los usuarios comunes deben ser capaces de ejecutar la orden **`passwd`** para modificar su clave, pero no se les debe permitir que modifiquen el fichero de cuentas de usuario directamente. Entonces, se activa el bit *set-uid* para el programa **`passwd`** (si en su sistema el programa **`passwd`** cuelga del directorio `/bin` ó en el directorio `/usr/bin` puede hacer `ls -l /bin/passwd` ó `ls -l /usr/bin/passwd` para comprobar la activación de dicho bit), con lo cual puede escribir en `/etc/shadow` (donde se guardan las claves cifradas) y en `/etc/passwd` a la vez que se niega el permiso a los usuarios comunes.

Nota: puede ejecutar la orden `whereis` para localizar un fichero, por ejemplo `whereis passwd`.

Algunos sistemas, entre ellos Linux, sólo reconocen el bit *set-uid* cuando aparece en los permisos de un programa compilado, y lo ignoran si aparece en los permisos de un guion de shell. Sin embargo, la utilización del bit *set-uid* no está limitada al superusuario.

Diferencia entre la s minúscula y la S mayúscula como bit SUID

¿Recuerdas la definición de SUID? Permite que un archivo se ejecute con los mismos permisos que el propietario del archivo. ¿Pero qué pasa si el archivo no tiene el bit de ejecución establecido en primer lugar? Así:

```
alumno@so-lab01:~$ ls -l prueba.txt
-rw-rw-r-- 1 alumno alumno 0 sep 28 16:37 prueba.txt
```

Si se activa el bit SUID, se mostrará una **S** mayúscula, no una **s** minúscula:

```
alumno@so-lab01:~$ chmod u+s prueba.txt
alumno@so-lab01:~$ ls -l prueba.txt
-rwSrw-r-- 1 alumno alumno 0 sep 28 16:37 prueba.txt
```

La bandera **S** como SUID **significa que hay un error que debes investigar**. Se quiere que el archivo se ejecute con el mismo permiso que el propietario, pero no hay permiso de ejecución en el archivo. Lo que significa que ni siquiera el propietario puede ejecutar el archivo. **El permiso set-uid (también set-gid) está ligado al de ejecución, ambos deben estar activos**.

El bit set-gid

El bit set-gid es parecido al bit set-uid, excepto que se aplica a los permisos de grupo en lugar de a los permisos de usuario. Cuando un programa o un directorio con el bit set-gid activado tiene acceso a un fichero, el programa utiliza los privilegios de grupo del fichero de programa o del directorio en lugar de los del grupo del usuario. El bit set-gid también se especifica con **`s`**, en el lugar del permiso de ejecución del grupo y se activa con el valor octal **2 para set-gid bit** en formato octal.

En Ubuntu, en el directorio `/var`, los permisos setgid en los directorios `local y mail` están activos para fines de seguridad y para simplificar la gestión de permisos para diferentes programas y usuarios.

Bit Adhesivo (Sticky bit)

El bit adhesivo, también llamado bandera de eliminación restringida, tiene el valor octal `1` y en modo simbólico está representado por una **`t`** dentro de los permisos de otros. Esto se aplica solo a los directorios y no tiene ningún efecto en los archivos normales. En Linux, evita que los usuarios eliminen o cambien el nombre de un archivo que se encuentra en un directorio con este permiso activo, a menos que sean propietarios de ese archivo o directorio.

Los directorios con el bit adhesivo establecido muestran una **`t`** reemplazando la `x` en los permisos de otros en la salida de **`ls -l`**:

```
alumno@so-lab01:~$ ls -ld directorio_ejemplo/
drwxr-xr-t 2 alumno alumno 4096 sep 28 16:37 directorio_ejemplo/
```

En el modo octal, los permisos especiales se especifican mediante una notación de 4 dígitos, donde el primer dígito representa el permiso especial para actuar. Por ejemplo, para establecer el bit adhesivo (valor 1) para el directorio `otro_directorio` en modo octal, con permisos `755`, el comando sería:

```
alumno@so-lab01:~$ chmod 1755 otro_directorio
alumno@so-lab01:~$ ls -ld otro_directorio
drwxr-xr-t 2 alumno alumno 4096 sep 28 16:37 otro_directorio
```

Este permiso se suele utilizar en el directorio `/tmp` que funciona como la papelera de los archivos temporales.

```
alumno@so-lab01:~$ ls -ld /tmp
drwxrwxrwt 6 root root 4096 sep 28 16:37 /tmp
```

Como puedes ver, la carpeta /tmp, tiene la letra t en el mismo lugar que esperamos ver x o – para otros permisos. Esto significa que un usuario (excepto root) no puede borrar los archivos temporales creados por otros usuarios en el directorio /tmp. En Ubuntu, los directorios que por defecto suelen tener el permiso "sticky bit" activado son /tmp, /var/tmp y /run/lock.

6.2. Permisos de directorios

Los **permisos** de ficheros **se aplican también a los directorios**, aunque **funcionan de una forma diferente**.

El permiso **`r`** de un directorio **permite averiguar (listar) qué hay en un directorio**, pero no es suficiente **para tener acceso a los ficheros cuyos nombres aparecen en él** (para ello también **hay que tener el permiso `x` activado** que nos permite atravesarlo). Por

ejemplo, no se puede leer el contenido de los ficheros de un directorio si sólo se tiene acceso **`r`** debe además tener activado el permiso **`x`**.

El permiso **`w`** de un directorio es necesario para **añadir ficheros o eliminarlos del directorio**, pero **NO para modificar un fichero** que aparece en el directorio (para ello habrá que tener el permiso **`w`** del fichero en cuestión). Para editar un fichero debemos poder llegar hasta él y además que éste tenga el permiso de escritura activado.

El permiso **x** de un **directorio** permite al usuario operar sobre los nombres del directorio, **alcanzar o atravesar dicho directorio** pero no le permite saber su contenido. Un directorio puede tener activado el permiso **`x`**, esto es, podemos pasar por él pero si no tiene además activado el permiso de lectura no podemos saber su contenido, esto es, no se puede listar su contenido. Sin el permiso **`x`** un usuario no puede ni ejecutar ni leer o modificar los ficheros de un directorio porque no se puede situar en ese directorio.

La última parte sobre los directorios puede sonar un poco confusa. Imaginemos, por ejemplo, que tienes un directorio llamado `solo_paso`, con los siguientes permisos:

```
alumno@so-lab01:~$ ls -ld solo_paso/
d--xr-xr-x 2 alumno alumno 4096 sep 28 16:37 solo_paso/
```

Imagina también que dentro de este directorio tienes un script de shell llamado `hola.sh` con los siguientes permisos:

```
alumno@so-lab01:~$ ls -l solo_paso/hola.sh
-rwxr-xr-x 1 alumno alumno 35 sep 28 16:37 solo_paso/hola.sh
```

Si eres el usuario `alumno` e intentas enumerar el contenido de `solo_paso`, recibirás un mensaje de error, ya que su usuario no tiene permiso de lectura para ese directorio:

```
alumno@so-lab01:~$ ls -l solo_paso/
ls: no se puede abrir el directorio 'solo_paso/': Permiso denegado
```

Sin embargo, el usuario `alumno` tiene permisos de ejecución, lo que significa que puede entrar o situarse en el directorio. Por lo tanto, el usuario alumno puede acceder a los archivos dentro del directorio, siempre que tenga los permisos correctos para el archivo respectivo. En este ejemplo, el usuario tiene permisos completos para el script `hola.sh`, por lo que puede ejecutar el script, incluso si no puede leer el contenido del directorio que lo contiene. Todo lo que se necesita es el nombre completo del archivo.

```
alumno@so-lab01:~$ sh solo_paso/hola.sh
Hola desde el guion
```

6.3. Establecimiento de permisos de acceso

La orden **`chmod`** permite cambiar los permisos de un fichero. Sólo puede hacerlo su propietario (o root); no se necesita permiso de escritura sobre el fichero ni sobre el directorio que lo contiene.

El formato de la orden **`chmod`** (*change mode*, cambiar modo) es:

```
$ chmod especifmodo fichero ...
```

donde cada `fichero` es la trayectoria de un fichero, y `especifmodo` es

- una lista de cambios de permisos separados por comas, o bien
- un número octal con un máximo de cuatro dígitos.

Cada cambio de permiso tiene tres partes: una o más letras de "quién", un operador, y una o más letras de permiso. Las letras de "quién" son:

- **`u`** permiso para el propietario del fichero, user.
- **`g`** permiso para el grupo del fichero, group.
- **`o`** permiso para todos los demás usuarios, others.
- **`a`** permiso para todos los usuarios (equivale a ugo), all.

Los operadores son:

- **`+`** añadir estos permisos
- **`-`** quitar estos permisos
- **`=`** establecer exactamente estos permisos, eliminando todos los demás para las letras "quién" implicadas.

Los permisos son:

- **`r`** lectura
- **`w`** escritura
- **`x`** ejecución
- **`s`** asignar set-uid o set-gid.
- **`t`** asignar sticky.

Cada cambio modifica los permisos de los ficheros en fichero ... de acuerdo con lo que se indique. Si se omiten las letras de "quién", se toma por omisión el valor `a` (all, todos), pero los permisos se modifican con el valor **`umask`** que veremos en la siguiente subsección.

Especificación de permisos en octal

Quizás la forma más sencilla de especificar los permisos sea utilizando un número octal de 4 dígitos como máximo, y si se escriben menos, se completan con ceros por la izquierda (chmod 7 fichero equivale a chmod 0007 fichero). Los permisos se pueden representar de forma numérica en un sistema octal, sumando los valores en binario de cada permiso:

- **Lectura (r)**: `100` **4** (2^2)
- **Escritura (w):** `010` **2** (2^1)
- **Ejecución (x)**: `001` **1** (2^0)

La combinación de permisos se obtiene sumando estos valores. Por ejemplo, `rwx` (lectura, escritura y ejecución) es `4+2+1=7`. `r-x` (lectura y ejecución) es `4+0+1=5`.

Permisos	Representación Octal	Ejemplo de Combinación
`---`	0	Ningún permiso
`--x`	1	Ejecución
`-w-`	2	Escritura
`-wx`	3	Escritura + Ejecución
`r--`	4	Lectura
`r-x`	5	Lectura + Ejecución
`rw-`	6	Lectura + Escritura
`rwx`	7	Lectura + Escritura + Ejecución

Para los permisos especiales, se añade un dígito al principio de la representación octal de tres dígitos, donde **4** es `setuid`, **2** es `setgid` y **1** es `sticky bit`. Por ejemplo, un fichero con permisos `rwsr-x--x` se representaría como `4751`.

4000 activar el bit set-uid.

2000 activar el bit set-gid.

1000 activar el bit adhesivo (sticky).

0400 asignar permiso de lectura para el propietario, r--------, 100 000 000.

0200 asignar permiso de escritura para el propietario, -w-------, 010 000 000.

0100 asignar permiso de ejecución para el propietario, --x------, 001 000 000.

0070 asignar permisos de lectura, escritura o ejecución para el grupo.

0007 asignar permisos de lectura, escritura o ejecución para otros usuarios.

Restricciones

- Únicamente el propietario del fichero o el superusuario pueden cambiar los permisos de un fichero.
- Únicamente puede activar el bit **s** para un grupo un usuario cuyo grupo sea el mismo que el grupo del fichero, y sólo si se permite al grupo la ejecución del fichero.
- El bit **s** no tiene sentido para el resto de los usuarios **o**.

Ejemplos

A continuación se presentan algunos ejemplos de utilización de **chmod**:

```
alumno@so-lab01:~$ chmod ug+wx,o+x sartre
```

otorga permisos de escritura y ejecución del fichero `sartre` al propietario y al grupo de `sartre`, y permiso de ejecución a los demás usuarios.

```
alumno@so-lab01:~$ chmod +x camus
```

suponiendo que el valor `umask es 0027`, confiere al propietario y a todos los usuarios de su grupo el permiso de ejecutar `camus`, pero no a los demás usuarios, ya que, al omitir las letras de «quién», el valor umask impide añadir los permisos que tiene activados (aunque no retira los que ya existían).

```
alumno@so-lab01:~$ chmod o=wx kafka
```

asigna los permisos de `kafka` para que otros usuarios puedan "escribir" y "ejecutar". Los permisos **u** y **g** no cambian.

```
alumno@so-lab01:~$ chmod 644 moloko
```

asigna los permisos de lectura y escritura al propietario de `moloko`, y el de lectura a los miembros del grupo y demás usuarios.

6.4. Reducción de los permisos para ficheros de nueva creación (umask)

Cuando un programa crea un fichero, especifica un conjunto de permisos para él. Los conjuntos usuales son rw-rw-rw- (666 en octal) para ficheros de datos y rwxrwxrwx (777 en octal) para directorios y ficheros ejecutables.

Los permisos de este conjunto se reducen después con el valor **umask** (mascarilla de usuario), que se puede cambiar con la orden **umask**. Los permisos se calculan realizando la operación AND bit a bit entre el valor de **umask**, negado bit a bit, y el valor especificado por el programa. Actúa como una máscara de bits que, por defecto, elimina ciertos permisos del conjunto predeterminado.

Así pues, si el valor de **umask** es `002` (permiso de escritura para otros usuarios), aquellos fuera de su grupo no podrán escribir en los ficheros que usted cree, a menos que cambie el modo de los ficheros con **chmod** o modifique su valor **umask**. Un valor umask característico es 022, con lo cual se niega el permiso de escritura a todos los usuarios menos usted. En Ubuntu, el valor por omisión en las sesiones de usuario es 0002 (umask lo muestra con cuatro dígitos).

La sintaxis de la orden umask es:

```
$ umask [n]
```

donde `n` es un número octal de hasta cuatro dígitos que especifica los permisos que se negarán, **`umask`** dice los permisos que queremos quitar. Si se omite **`n`**, `umask` muestra el valor `umask` vigente. Una vez que se ha asignado el valor `umask`, sigue vigente durante la ejecución de su shell o hasta que se cambie de nuevo.

Realiza el siguiente ejercicio:

1. Pon una máscara a 000

```
alumno@so-lab01:~$ umask 000
```

2. Crea un fichero y un directorio

```
alumno@so-lab01:~$ touch fichero
alumno@so-lab01:~$ mkdir directorio
```

3. Mira los permisos que tienen

```
alumno@so-lab01:~$ ls -l
```

¿Por qué los permisos son diferentes para el fichero y para el directorio?

6.5. Ejemplos

En este apartado se pretende aclarar el uso y funcionamiento de los permisos mediante algunos ejemplos. En ellos se utilizarán las siguientes órdenes:

- **cat**: cuando se utiliza con la sintaxis **`cat fich`**, `cat` intenta abrir el fichero fich en modo de sólo lectura. Si tiene permisos para ello lo lee, y muestra su contenido en la salida estándar.
- **cambia: su sintaxis de uso es ./cambia fich c1 c2**. Este programa intenta abrir el fichero `fich` en modo de lectura y escritura. Si consigue hacerlo cambia en `fich` todas las ocurrencias del carácter `c1` por `c2`.
- **su:** cuando se utiliza con la sintaxis **`su u`** se ejecuta un intérprete de órdenes con el identificador de usuario, y de grupo, del usuario `u`. La orden nos pide la contraseña de u. Para más información consultar el manual.
- **Para crear un usuario normal (debe ser administrador del sistema)**

  Si se ejecuta adduser sin opciones, creará un usuario normal. Veamos un ejemplo:

  ```
  $ sudo adduser nombre_usuario
  ```

  La orden anterior creará un usuario con las siguientes **opciones por defecto**:

  - Se le asigna y creará el directorio de inicio en `/home/nombre_usuario`.

    - Se le asigna la shell `/bin/bash`.
    - Pregunta por su nombre completo y otros datos como el teléfono, etc.
    - Se crea el grupo de nombre igual al nombre del usuario.
    - Pregunta por la contraseña para asignarle una. Al introducir la contraseña esta no es visible.
- Descarga el programa cambia.cpp que aparece en la Introducción de este tema y compílalo con g++, como se muestra en el primer ejemplo (el compilador ya crea el ejecutable con permiso de ejecución).

Los ejemplos necesitan un segundo usuario, alumno2, que puede crearse como se acaba de indicar (`sudo adduser alumno2`). Además, en Ubuntu cada directorio base se crea con permisos rwxr-x--- (750), de modo que los demás usuarios no pueden atravesarlo: para que alumno2 llegue hasta d/f, antes de cambiar de usuario se concede el permiso x a los otros con `chmod o+x ~`, y se retira al terminar con `chmod o-x ~`.

Ejecutemos ahora los ejemplos:

```
alumno@so-lab01:~$ cd  # Nos desplazamos a nuestro directorio base
alumno@so-lab01:~$ cp ~/Descargas/cambia.cpp .  # Copiamos cambia.cpp a nuestro
directorio base
alumno@so-lab01:~$ g++ -std=c++17 -Wall -Wextra cambia.cpp -o cambia  #
Compilamos el programa
alumno@so-lab01:~$ umask 022  # Establecemos esta máscara
alumno@so-lab01:~$ mkdir d  # Creamos el directorio d
alumno@so-lab01:~$ ls -ld d  # Consultamos sus permisos
drwxr-xr-x 2 alumno alumno 4096 sep 28 16:38 d
alumno@so-lab01:~$ echo murcielago > d/f  # Creamos d/f; sin tilde, cada letra
ocupa un byte
alumno@so-lab01:~$ ls -l d/f  # Observamos con qué permisos se creó d/f
-rw-r--r-- 1 alumno alumno 11 sep 28 16:38 d/f
alumno@so-lab01:~$ cat d/f  # Observamos (leemos) el contenido de d/f
murcielago
alumno@so-lab01:~$ ./cambia d/f a A  # Cambiamos las apariciones de a por A en
d/f
✅ Reemplazo completado con éxito en el fichero 'd/f'.
alumno@so-lab01:~$ cat d/f  # Comprobamos que los cambios se realizan
murcielAgo
```

```
alumno@so-lab01:~$ chmod 000 d/f  # Cambiamos los permisos de d/f
alumno@so-lab01:~$ ls -l d/f  # Observamos los permisos de d/f
---------- 1 alumno alumno 11 sep 28 16:38 d/f
alumno@so-lab01:~$ cat d/f  # Ahora no podemos leer d/f
cat: d/f: Permiso denegado
alumno@so-lab01:~$ ./cambia d/f e E  # Ni modificarlo
Error: No se pudo abrir el fichero 'd/f'
```

```
alumno@so-lab01:~$ chmod 644 d/f  # Restablecemos los permisos de d/f
alumno@so-lab01:~$ chmod 100 d  # Establecemos los permisos --x------ en d
alumno@so-lab01:~$ cat d/f  # Podemos acceder a los ficheros de d si sabemos su
nombre
murcielAgo
alumno@so-lab01:~$ ls -l d  # Pero no consultar su contenido, porque no se puede
leer d
ls: no se puede abrir el directorio 'd': Permiso denegado
alumno@so-lab01:~$ chmod 400 d  # Establecemos los permisos r-------- en d
alumno@so-lab01:~$ ls d  # Podemos consultar el contenido de d
ls: no se puede acceder a 'd/f': Permiso denegado
f
alumno@so-lab01:~$ cat d/f  # Pero no podemos usar ningún fichero que cuelgue de
d
cat: d/f: Permiso denegado
```

```
alumno@so-lab01:~$ chmod 755 d  # Restablecemos los permisos rwxr-xr-x de d
alumno@so-lab01:~$ chmod o+x ~  # Los demás usuarios pueden atravesar nuestro
directorio base
alumno@so-lab01:~$ su alumno2  # Ejecutamos un intérprete con otro usuario
Contraseña:
alumno2@so-lab01:/home/alumno$ cat d/f  # Podemos leer d/f siendo alumno2
murcielAgo
alumno2@so-lab01:/home/alumno$ ./cambia d/f i I  # Pero no modificar su contenido
Error: No se pudo abrir el fichero 'd/f'
alumno2@so-lab01:/home/alumno$ rm d/f  # Intentamos borrar d/f siendo alumno2
rm: ¿borrar el fichero regular 'd/f'  protegido contra escritura? (s/n) s
rm: no se puede borrar 'd/f': Permiso denegado
alumno2@so-lab01:/home/alumno$ exit  # Volvemos a ser el usuario alumno
exit
alumno@so-lab01:~$ chmod 666 d/f  # Establecemos los permisos rw-rw-rw- en d/f
alumno@so-lab01:~$ su alumno2  # Volvemos a ejecutar un intérprete con alumno2
Contraseña:
alumno2@so-lab01:/home/alumno$ ./cambia d/f i I
✅ Reemplazo completado con éxito en el fichero 'd/f'.
alumno2@so-lab01:/home/alumno$ cat d/f
murcIelAgo
alumno2@so-lab01:/home/alumno$ exit  # Volvemos a ser el usuario alumno
exit
alumno@so-lab01:~$ chmod 777 d  # Establecemos los permisos rwxrwxrwx en d
alumno@so-lab01:~$ su alumno2  # Volvemos a ejecutar un intérprete con alumno2
Contraseña:
alumno2@so-lab01:/home/alumno$ rm d/f  # Borramos d/f aun siendo alumno2
alumno2@so-lab01:/home/alumno$ exit  # Volvemos a ser el usuario alumno
exit
```

```
alumno@so-lab01:~$ umask  # Comprobamos el valor de umask
0022
alumno@so-lab01:~$ mkdir d2  # Creamos un directorio d2
alumno@so-lab01:~$ ls -ld d2  # Consultamos sus permisos
drwxr-xr-x 2 alumno alumno 4096 sep 28 16:38 d2
alumno@so-lab01:~$ echo basura > f  # Creamos el fichero f
alumno@so-lab01:~$ ls -l f  # Observamos con qué permisos se creó f
-rw-r--r-- 1 alumno alumno 7 sep 28 16:38 f
alumno@so-lab01:~$ umask 000  # Establecemos una máscara que no disminuye
permisos
alumno@so-lab01:~$ mkdir d3  # Creamos un directorio d3
alumno@so-lab01:~$ ls -ld d3  # Consultamos sus permisos
drwxrwxrwx 2 alumno alumno 4096 sep 28 16:38 d3
```

... como se observa, mkdir intenta crear directorios con permisos `rwxrwxrwx`

```
alumno@so-lab01:~$ echo basura > f2  # Creamos el fichero f2
alumno@so-lab01:~$ ls -l f2  # Observamos con qué permisos se creó f2
-rw-rw-rw- 1 alumno alumno 7 sep 28 16:38 f2
```

... cuando redireccionamos la salida estándar de un programa a un fichero que no existe utilizando el metacarácter ">" el shell intenta crear el fichero de salida con permisos `rw-rw-rw-`

```
alumno@so-lab01:~$ umask 0002  # Restablecemos la máscara por omisión de Ubuntu
alumno@so-lab01:~$ chmod o-x ~  # Restablecemos los permisos del directorio base
(rwxr-x---)
```

7. Ejercicios propuestos

1. Ejecuta una orden que:
    - Muestra el contenido del directorio actual con información extendida o larga de permisos.
    - Muestra los permisos del directorio actual, no de su contenido.
    - Crea los directorios público, privado y compartido en tu directorio personal.
    - Muestra los permisos de los tres directorios creados en tu directorio personal, público, privado y compartido, no de su contenido. Solo esos tres directorios.
    - Configura los permisos del directorio público para que el propietario tenga permisos de acceso, lectura y escritura, el grupo solo pueda acceder para leer y los otros usuarios no puedan realizar ninguna acción.
    - Configura los permisos del directorio privado para que el propietario tenga permisos de acceso, lectura y escritura, el grupo y el resto no puedan realizar ninguna acción.
    - Configura los permisos del directorio compartido para que el propietario y el grupo tengan permisos de acceso, lectura y escritura y los otros usuarios solo puedan acceder para leer.
    - Muestra información extendida de estos directorios.

2. Cree en su directorio base un directorio de nombre `d`, y dentro de este directorio un fichero de nombre `f` cuyo contenido sea la palabra `hola` y un fichero `f2` cuyo contenido sea la palabra `adiós`. Establezca permisos para `d` y `f` de forma que:
    - Usted pueda consultar el contenido de `d`, pero no pueda utilizar (no tenga acceso a) los ficheros que cuelgan de `d`.
    - Usted pueda consultar el contenido de `d` y pueda utilizar (tenga acceso a) los ficheros que cuelgan de `d`. Sin embargo, podrá leer el contenido de `f`, pero no el de `f2`.

- Usted tenga acceso a los ficheros que cuelgan de `d`, pero no pueda ver su contenido. (Sólo podrá acceder a ellos si sabe sus nombres)
- Usted pueda eliminar los ficheros que cuelgan de `d`, pero no puede leerlos ni modificar su contenido.
- Usted no pueda eliminar los ficheros que cuelgan de `d`, aunque sí pueda leerlos o modificar su contenido.

3. Normalmente un fichero tiene asignados los permisos de forma que se da más o igual privilegio de acceso al propietario que a los miembros del grupo, y más o igual privilegio a los miembros del grupo que al resto de los usuarios. Sin embargo, un fichero podría tener permisos de, por ejemplo, -wxrwxrwx. ¿Podrá el propietario del fichero leerlo? Como propietario no debería, pero, ¿y cómo miembro de grupo?, ¿y cómo miembro del resto?. Experimente con ello y deduzca qué hace UNIX en estos casos.
4. Indique justificadamente qué máscara debería suministrarle a la orden `umask` para que sus ficheros ordinarios se crearan a lo sumo con los permisos rwxr-x---.
5. Descargue del repositorio de la asignatura (enlace en la Introducción) el programa fuente **consultar.cpp** a su directorio base. El objetivo de este programa es consultar un fichero de texto compuesto de líneas con el siguiente formato:

```
login:nota:
```

Cada línea almacena dos campos (se utiliza el carácter : para separarlos): el primero representa el login de un usuario, y el segundo una nota (un valor entre 0 y 10). Un ejemplo de contenido de un fichero de este tipo es:

```
alumno:7:
alumno2:8.5:
```

El programa busca la línea que contiene el login del usuario que ejecuta el programa, y muestra su nota. Si el usuario no se encuentra en el fichero se muestra un mensaje alusivo en la salida estándar.

En este ejercicio debe crear en su directorio base un fichero de notas, llámelo, por ejemplo *`notas`*. Debe editar su fichero consultar.cpp y cambiar la línea

```
const std::string nomFich="/home/usuario/notas";
```

por una que refleje su fichero de notas, por ejemplo

```
const std::string nomFich="/home/alumno/notas";
```

Compile el programa, y póngale de nombre *`consultar`*. Asegúrese de que en el fichero de `notas` aparezca una entrada para el usuario con el que esté conectado al sistema. Establezca permisos de forma que los demás usuarios puedan saber su nota ejecutando `consultar`, pero no directamente visualizando el contenido de notas con un programa como cat, esto último les permitiría ver las notas de los demás usuarios. Los demás usuarios tampoco deberían poder modificar su nota.

Como resultado del ejercicio debe comentar qué permisos asignó, y por qué.

6. Ejecute una orden para saber cuántos dispositivos de carácter y cuantos dispositivos de bloque tenemos en nuestro sistema de ficheros. Utilice una sola orden utilizando una interconexión o pipe. Use las órdenes **`ls`**, **`grep`**, **`wc`**.

Preguntas frecuentes

Caracteres “comodín” ¿Qué son los caracteres "comodín"?

Una característica importante de la mayoría de los intérpretes de comandos en Unix es la capacidad para referirse a más de un archivo usando patrones (que no son expresiones regulares, aunque se parezcan). En su forma más simple, esto consiste en la utilización de los llamados “comodines” que permiten referirse a, por ejemplo, todos los archivos cuyo nombre contiene una determinada secuencia de caracteres.

Los comodines “**`*`**” y “**`?`**”

El comodín “**`*`**” hace referencia a cualquier cadena de caracteres en el nombre del archivo. Cuando se usa el carácter “**`*`**” para referirse al nombre de un archivo, el intérprete de comandos lo sustituye por todas las combinaciones posibles provenientes de los archivos en el directorio al cual nos estamos refiriendo.

Veamos un ejemplo rápido. Supongamos que **`diego`** tiene los archivos **`hugo`**, **`paco`** y **`luis`** en el directorio actual.

```
diego@so-lab01:~$ ls
hugo  luis  paco
diego@so-lab01:~$
```

Para listar todos los archivos con la letra “**`o`**” en su nombre, usamos la orden:

```
diego@so-lab01:~$ ls *o*
hugo  paco
diego@so-lab01:~$
```

Como podemos ver, el comodín “**`*`**” ha sido sustituido con todas las combinaciones posibles que coincidían de entre los archivos del directorio actual.

El uso de “**`*`**” solo, hace referencia a todos los archivos, puesto que todos los caracteres coinciden con el comodín.

```
diego@so-lab01:~$ ls *
hugo  luis  paco
diego@so-lab01:~$
```

Veamos algunos otros ejemplos:

```
diego@so-lab01:~$ ls h*
hugo
diego@so-lab01:~$ ls *is
luis
diego@so-lab01:~$ ls *u*
hugo  luis
diego@so-lab01:~$ ls p*o
paco
diego@so-lab01:~$
```

El proceso de la sustitución de caracteres como "*****" en nombres de archivos es llamado "expansión de comodines" y es efectuado por el intérprete de comandos. Esto es importante: los comandos, como **ls**, nunca ven el "*****" en su lista de parámetros. Es el intérprete quien expande los comodines para incluir todos los nombres de archivos que se adaptan. Por lo tanto la orden:

```
diego@so-lab01:~$ ls *o*
```

es expandida para obtener

```
diego@so-lab01:~$ ls hugo paco
```

Otro carácter comodín es "**?**". Este carácter comodín sólo expande un único carácter. Luego "**ls ?**" mostrará todos los nombres de archivos con un carácter de longitud, y "**ls termca?**", por ejemplo, mostrará "**termcap**" pero no "termcap.backup". Aquí tenemos otro ejemplo:

```
diego@so-lab01:~$ ls hu?o
hugo
diego@so-lab01:~$ ls p??o
paco
diego@so-lab01:~$ ls ???s
luis
diego@so-lab01:~$
```

Como podemos ver, los caracteres comodín nos permiten referirse a más de un archivo a la vez. Los comandos básicos como cp y mv pueden copiar o mover múltiples archivos de una vez. Por ejemplo:

```
diego@so-lab01:~$ cp /etc/s* /home/diego
```

copiará todos los archivos de **/etc** que comiencen con "**s**" al directorio **/home/diego**. En la práctica, cp omite los subdirectorios (como /etc/security) e informa de los ficheros que no puede leer (como /etc/shadow).

Octal ¿Qué es el sistema OCTAL?

Es el sistema numérico en base 8 se llama octal y utiliza los dígitos 0 a 7.

Para convertir un número en base decimal a base octal se divide por 8 sucesivamente hasta llegar a cociente 0, y los restos de las divisiones en orden inverso indican el número en octal. Para pasar de base 8 a base decimal, solo hay que multipicar cada cifra por 8 elevado a la posición de la cifra, y sumar el resultado.

Es más fácil pasar de binario a octal, porque solo hay que agrupar de 3 en 3 los dígitos binarios, así, el número 74 (en decimal) es 1001010 (en binario), lo agruparíamos como 1 / 001 / 010, después obtenemos el número en decimal de cada uno de los números en binario obtenidos: 1=1, 001=1 y 010=2. De modo que el número decimal 74 en octal es 112.

En informática, a veces se utiliza la numeración octal en vez de la hexadecimal, y se suele indicar poniendo un 0 delante del número octal (0x indica hexadecimal). Tiene la ventaja de que no requiere utilizar otros símbolos diferentes de los dígitos. Sin embargo, para trabajar con bytes o conjuntos de ellos, asumiendo que un byte es una palabra de 8 bits, suele ser más cómodo el sistema hexadecimal, por cuanto todo byte así definido es completamente representable por dos dígitos hexadecimales.